



UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Music Artists & Albums Public Perception

Workshop #3 — Gold Layer, Governance and Storytelling

Systems Engineering Curriculum
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia

Course: Programming for Data Analysis

Professor: Carlos Andres Sierra Virguez

May 30th

Carlos Andres Celis Herrera	20222020051
Juan Diego Lozada Gonzalez	20222020014
Cristian Santiago Lopez Cadena	20222020027

Contents

1	Executive Summary	3
2	Pipeline Improvements	3
2.1	Improvement 1 — Docker Compose: Automatic <code>fs_default</code> Connection	3
2.1.1	Observation	3
2.1.2	Change Applied	4
2.1.3	Justification	4
2.2	Improvement 2 — Docker Compose: Unified Volume Mounts and PySpark Image . .	4
2.2.1	Observation	4
2.2.2	Change Applied	4
2.2.3	Justification	5
2.3	Improvement 3 — <code>lastfm_silver</code> : Schedule and Bronze Read Strategy	5
2.3.1	Observation	5
2.3.2	Change Applied — Schedule Decoupling	5
2.3.3	Change Applied — Historical Read	5
2.3.4	Justification	6
2.4	Improvement 4 — <code>lastfm_silver</code> : Date-Aware Deduplication Key	6
2.4.1	Observation	6
2.4.2	Change Applied	6
2.4.3	Justification	6
2.5	Improvement 5 — Silver: Single Consolidated Parquet File	6
2.5.1	Observation	6
2.5.2	Change Applied	6
2.5.3	Justification	7
2.6	Improvement 6 — <code>reddit_silver</code> : Multi-File Read and Stable Deduplication	7
2.6.1	Observation	7
2.6.2	Change Applied	7
2.6.3	Justification	7
2.7	Summary of Improvements	8
3	Data Quality Findings	8
3.1	Volume and Schema Overview	8
3.2	Null Rate Analysis	9
3.2.1	Reddit	9
3.2.2	Last.fm	9
3.3	Descriptive Statistics — Numeric Fields	9
3.3.1	Reddit	9
3.3.2	Last.fm	10
3.4	Outlier Analysis (IQR Method)	10
3.5	Text Length Distributions	11
3.6	NLP Classification Findings	11

4	Governance KPI Definition	11
4.1	Rationale	11
4.2	KPI Catalogue	12
4.3	Gold Governance Schema	12
4.4	Observed KPI Values (Current Run)	13
5	Storytelling Aggregations	13
5.1	Rationale and User Stories	13
5.2	Metric Catalogue	14
5.3	Gold Storytelling Schema	15
5.4	Observed Storytelling Values (Current Run)	15
5.4.1	Sentiment Distribution	15
5.4.2	Sentiment Trend	15
5.4.3	Comment Type Distribution	15
5.5	Dashboard Narrative	15
6	PySpark Implementation	16
6.1	Architecture	16
6.2	SparkSession Configuration	16
6.3	Governance Computation	16
6.4	Sentiment Analysis with VADER	17
6.5	Keyword Analysis	17
6.6	Output Persistence	18
6.7	Docker Requirements for PySpark	18
7	Results Evidence	19
7.1	Pipeline Output Files	19
7.2	Airflow DAG Execution Summary	19
7.3	Key Analytical Results	19
7.4	Notebooks	20

1 Executive Summary

Workshop #3 extends the Medallion Architecture pipeline established in Workshop #2 by introducing the **Gold layer**, a **data governance framework**, and a **storytelling summary** designed to populate a functional dashboard for music label analysts and artist managers.

The work delivered in this workshop covers four main areas:

- **Pipeline improvements** — Six targeted changes to the Bronze and Silver DAGs to correct data consolidation, scheduling, and infrastructure issues identified after the initial Workshop #2 deployment.
- **Data quality findings** — A structured analysis of the Silver layer across three sources (Reddit, Last.fm Top Artists, Last.fm Top Tracks), documenting null rates, outlier rates, text length distributions, classification results and schema compliance.
- **Governance KPIs** — Nine KPI types computed with PySpark over all Silver Parquet files and persisted as `governance_*.parquet` in the Gold layer.
- **Storytelling aggregations** — Eight metric types computed with PySpark and VADER sentiment analysis and persisted as `storytelling_*.parquet`, providing the analytical foundation for the dashboard.

Key finding: Of the 228 Reddit comments analysed, **67.98% express positive sentiment** (average VADER compound score: +0.621), **14.04% are negative** (−0.524), and **17.98% are neutral**. The indie and hip-hop community reacts with genuine enthusiasm to new releases, but discusses albums as a concept rather than individual songs — a critical insight for campaign planning.

2 Pipeline Improvements

This section summarizes each improvement made to the Bronze and Silver processes since their initial implementation in Workshop #2. For each change we show the before/after comparison and the rationale.

2.1 Improvement 1 — Docker Compose: Automatic fs_default Connection

2.1.1 Observation

After the initial Docker stack deployment, the `FileSensor` operators in both Silver DAGs failed on their first run:

```
airflow.exceptions.AirflowNotFoundException:  
The conn_id 'fs_default' isn't defined
```

The connection had to be created manually through the Airflow UI after every `docker compose up`.

2.1.2 Change Applied

The `airflow-init` entrypoint now creates `fs_default` automatically.

Before:

```
airflow db migrate &&
airflow users create --username ... || true
```

After:

```
airflow db migrate &&
airflow users create --username ... || true &&
airflow connections add fs_default \
  --conn-type fs \
  --conn-extra '{"path": "/"}' || true
```

2.1.3 Justification

The `|| true` flag ensures idempotency across restarts. The environment is now fully self-configuring with a single `docker compose up`.

2.2 Improvement 2 — Docker Compose: Unified Volume Mounts and PySpark Image

2.2.1 Observation

Volume mounts were inconsistent across services: `datalake_silver` and `datalake_gold` were missing from several containers, causing `FileNotFoundError` in Silver and Gold DAGs. The base image also lacked Java and PySpark, required by the Gold pipeline.

2.2.2 Change Applied

A custom `Dockerfile` was created extending `apache/airflow:2.8.1` with OpenJDK 17, PySpark 3.5.1 and vaderSentiment 3.3.2. All volume mounts were consolidated in the `x-airflow-common` anchor:

```
# Dockerfile
FROM apache/airflow:2.8.1
USER root
RUN apt-get update && apt-get install -y openjdk-17-jdk-headless
ENV JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
USER airflow
RUN pip install pyspark==3.5.1 vaderSentiment==3.3.2
```

```
# docker-compose.yml anchor (inherited by all services)
x-airflow-common: &airflow-common
  build: .
  volumes:
    - ./datalake_bronze:/opt/airflow/datalake_bronze
    - ./datalake_silver:/opt/airflow/datalake_silver
    - ./datalake_gold:/opt/airflow/datalake_gold
```

2.2.3 Justification

Centralising mounts in the anchor eliminates the risk of a future service being added without the required paths. Building from a custom image guarantees Java, PySpark and VADER are available in every Airflow container.

2.3 Improvement 3 — lastfm_silver: Schedule and Bronze Read Strategy

2.3.1 Observation

lastfm_silver ran @daily, triggered automatically by the Bronze DAG via TriggerDagRunOperator. Each run processed only the most recent Bronze file, discarding all accumulated historical snapshots.

2.3.2 Change Applied — Schedule Decoupling

Before:

```
1 trigger_silver = TriggerDagRunOperator(
2     task_id="trigger_silver_dag",
3     trigger_dag_id="lastfm_silver",
4 )
5 summary >> trigger_silver
6
7 @dag(dag_id="lastfm_silver", schedule="@daily", ...)
```

After:

```
1 # Bronze DAG    no longer triggers Silver
2 validate_bronze_files(tracks_path, artists_path)
3
4 # Silver DAG    independent weekly schedule
5 @dag(dag_id="lastfm_silver", schedule="@weekly", ...)
```

2.3.3 Change Applied — Historical Read

Before:

```
1 def _get_latest_bronze_file(folder):
2     files = sorted(glob.glob(os.path.join(folder, "*.json")))
3     return files[-1]    # only the most recent file
```

After:

```
1 def _read_all_bronze_files(folder, data_key, record_key):
2     files = sorted(glob.glob(os.path.join(folder, "*.json")))
3     all_records = []
4     for filepath in files:
5         with open(filepath, "r", encoding="utf-8") as f:
6             raw = json.load(f)
7             ingested_at = raw["_metadata"]["ingested_at"]
8             records = raw["data"].get(data_key, {}).get(record_key, [])
9             for r in records:
10                 r["_ingested_at"] = ingested_at
11             all_records.extend(records)
12     return all_records
```

2.3.4 Justification

Consolidating all daily Bronze snapshots allows the Gold layer to compute temporal trend aggregations (e.g. listener count growth across days). A single snapshot provides no trend information.

2.4 Improvement 4 — lastfm_silver: Date-Aware Deduplication Key

2.4.1 Observation

Deduplication by `name` alone discarded all but one temporal snapshot per artist, eliminating the time series.

2.4.2 Change Applied

Before:

```
1 df, dupes = deduplicate(df, key="name")
2 # Result: one row per artist, no temporal dimension
```

After:

```
1 df["_dedup_key"] = (
2     df["name"] + "||" +
3     pd.to_datetime(df["ingested_at"]).dt.date.astype(str)
4 )
5 df, dupes = deduplicate(df, key="_dedup_key")
6 df = df.drop(columns=["_dedup_key"])
7 # Result: one row per artist per day      temporal snapshots preserved
```

2.4.3 Justification

The deduplication key `name + date` uniquely identifies an artist on a given day, removing true intra-day duplicates while preserving the temporal evolution required by the Gold storytelling aggregations.

2.5 Improvement 5 — Silver: Single Consolidated Parquet File

2.5.1 Observation

Each Silver run produced a new timestamped Parquet file, accumulating 7+ small files per source per week and complicating PySpark reads in the Gold layer.

2.5.2 Change Applied

Both Last.fm Silver DAGs and the Reddit Silver DAG now produce a single authoritative file per source, overwriting it on each weekly run.

Before:

```
1 timestamp      = datetime.utcnow().strftime("%Y%m%d_%H%M%S")
2 filepath_out = _save_parquet(df, schema, path,
3     f"lastfm_top_artists_{timestamp}.parquet")
```

After:

```
1 filepath_out = _save_parquet(df, schema, path,
2     "lastfm_top_artists_current.parquet")
```

2.5.3 Justification

Since the Silver read function already consolidates all Bronze history, there is no need to accumulate multiple Silver files. A single file ensures the Gold PySpark read always finds one authoritative dataset.

2.6 Improvement 6 — reddit_silver: Multi-File Read and Stable Deduplication

2.6.1 Observation

The Reddit Silver DAG processed only the most recent Bronze JSON, discarding all previously scraped data. The deduplication key relied on `post_id`, a local DataFrame index that resets on each run and is therefore not stable across files.

2.6.2 Change Applied

The pipeline now reads all Bronze Reddit JSON files and uses a composite deduplication key stable across scraping sessions:

Before:

```
1 df = df.drop_duplicates(subset=["post_id", "clean_comment"])
```

After:

```
1 df["_dedup_key"] = (  
2     df["title"] + "||" +  
3     df["clean_comment"] + "||" +  
4     df["_ingested_at"].astype(str)  
5 )  
6 df = df.drop_duplicates(subset=["_dedup_key"]).drop(columns=["_dedup_key"])
```

2.6.3 Justification

`title` is stable across scraping sessions, unlike the local `post_id` index. The key `title + clean_comment + date` uniquely and reproducibly identifies a comment across multiple Bronze files.

2.7 Summary of Improvements

Table 1: Summary of pipeline improvements — Workshop #2 → Workshop #3

Component	Before	After
Docker Compose	<code>fs_default</code> created manually after each <code>docker compose up</code> .	Created automatically in <code>airflow-init</code> entrypoint.
Docker Image	<code>apache/airflow:2.8.1</code> — no Java, no PySpark.	Custom image with OpenJDK 17, PySpark 3.5.1 and vaderSentiment 3.3.2.
<code>lastfm_ingest</code>	Triggered Silver via <code>TriggerDagRunOperator</code> .	Decoupled; Silver runs on its own <code>@weekly</code> schedule.
<code>lastfm_silver</code>	<code>@daily</code> ; read only the latest Bronze JSON.	<code>@weekly</code> ; reads all accumulated Bronze files.
<code>lastfm_silver</code>	Deduplication by <code>name</code> only — lost temporal data.	Deduplication by <code>name + date</code> — preserves time series.
Silver (both)	New timestamped Parquet file per run.	Single <code>_current.parquet</code> overwritten on each weekly run.
<code>reddit_silver</code>	Most recent Bronze JSON only; <code>post_id</code> -based deduplication.	All Bronze files; stable <code>title + clean_comment + date</code> key.

3 Data Quality Findings

This section documents all data quality observations found in the Silver layer across the three sources analysed. Findings were obtained by running the `notebooks/data_quality.ipynb` notebook directly against the Silver Parquet files using Pandas.

3.1 Volume and Schema Overview

Table 2: Silver layer volume and schema compliance by source

Source	Rows	Columns	Schema Compliance	Ingestion Dates
Reddit comments	228	19	100%	1
Last.fm Top Artists	650+	6	100%	13
Last.fm Top Tracks	650+	10	100%	13

Schema compliance is 100% across all sources because the Silver pipeline enforces the PyArrow schema and raises a `ValueError` before writing if any required field contains nulls. This is by design: the Gold layer assumes required fields are always non-null.

3.2 Null Rate Analysis

3.2.1 Reddit

All 16 required fields have a null rate of **0.0%**. The three optional fields show the following rates of the sentinel value "unknown":

Table 3: Reddit silver — optional field fill rate

Field	Note	Rate → “unknown”
artist	Extracted from comment	≈ 95%
song	Extracted from comment	≈ 95%
pattern_type	Music pattern classifier	≈ 99%

The high "unknown" rate in **artist** and **song** reflects that most Reddit comments are free-form opinions without the structured format **Artist | Song** or **Song by Artist** that the extraction regex requires. This is an expected data characteristic, not a pipeline defect.

3.2.2 Last.fm

Table 4: Last.fm silver — optional field fill rate

Source	Field	Note	Rate → “unknown”
Artists	mbid	Not all entities registered in MusicBrainz	≈ 30–40%
Tracks	mbid	Track MusicBrainz ID	≈ 25–35%
Tracks	artist_mbid	Artist MusicBrainz ID on the track record	≈ 30–40%

The **mbid** fill rate is acceptable: MusicBrainz coverage is incomplete by design, and the field is not used in any downstream KPI or aggregation.

3.3 Descriptive Statistics — Numeric Fields

3.3.1 Reddit

Table 5: Reddit silver — descriptive statistics for numeric fields

Field	Mean	Std	Min	Median	Max
score	64.9	122.5	0	15.0	607
word_count	11.6	14.2	1	8.0	143
confidence	0.17	0.14	0.00	0.15	0.45
extract_confidence	0.02	0.10	0.00	0.00	1.00

3.3.2 Last.fm

Table 6: Last.fm silver — descriptive statistics for numeric fields

Field	Mean	Min	Median	Max
playcount (artists)	452M	18M	270M	4,122M
listeners (artists)	2.4M	270K	1.8M	6.8M
playcount (tracks)	58M	1M	25M	702M
listeners (tracks)	510K	15K	350K	3.3M
duration_sec (tracks)	218s	44s	210s	630s

3.4 Outlier Analysis (IQR Method)

Outliers are defined as values outside $[Q_1 - 1.5 \cdot \text{IQR}, Q_3 + 1.5 \cdot \text{IQR}]$. Table 7 summarises the outlier rate per numeric field.

Table 7: IQR outlier rates per numeric field

Source	Field	N Outliers	Outlier Rate
Reddit	score	≈ 34	$\approx 15\%$
Reddit	word_count	≈ 23	$\approx 10\%$
Reddit	confidence	≈ 5	$\approx 2\%$
Artists	playcount	≈ 6	$\approx 12\%$
Artists	listeners	≈ 4	$\approx 8\%$
Tracks	playcount	≈ 8	$\approx 16\%$
Tracks	duration_sec	≈ 2	$\approx 4\%$

Key observations:

- **reddit.score**: Viral posts (score > 500) inflate the mean to 64.9 against a median of 15. IQR capping ($\rightarrow$ **score_capped**) reduces the effective range from 607 to approximately 200 without removing any rows.
- **reddit.word_count**: Long recommendation threads (up to 143 words) are statistical outliers. Capping ($\rightarrow$ **word_count_capped**) normalises the metric for aggregations.
- **artists.playcount**: The distribution is heavily right-skewed (BTS: 4.12B plays vs. median 270M). These values are *correct* and are not capped; they are used only in relative rankings.

3.5 Text Length Distributions

Table 8: Text length statistics by field (characters)

Source	Field	Mean	Median	Min	Max
Reddit	<code>raw_comment</code>	≈ 280	≈ 120	3	3,200
Reddit	<code>clean_comment</code>	≈ 220	≈ 90	2	2,800
Reddit	<code>word_count</code>	11.6 tokens	8 tokens	1	143
Artists	<code>name</code>	≈ 10	≈ 8	2	30
Tracks	<code>name</code>	≈ 12	≈ 9	1	45

The reduction from `raw_comment` to `clean_comment` (mean: -60 chars, median: -30 chars) confirms that the NLP pipeline successfully removes HTML entities, URLs and excess punctuation introduced by Reddit’s markdown rendering.

3.6 NLP Classification Findings

Table 9: Reddit silver — comment type distribution

Comment Type	Count	Percentage
<code>opinion</code>	179	78.5%
<code>other</code>	48	21.1%
<code>mixed</code>	1	0.4%
<code>recommendation</code>	0	0.0%
Total	228	100%

Finding: Zero `recommendation` comments were detected across all 228 records. Users on `r/indieheads` and `r/hiphopheads` write in discursive, unstructured prose when reacting to new album releases. The structured patterns (`Artist | Song`, `Song by Artist`, `Artist: Song`) that the classifier detects do not appear in first-listen reaction threads. This finding shifts the analytical focus from entity extraction to **sentiment analysis**.

4 Governance KPI Definition

4.1 Rationale

The governance KPIs were selected based on the data quality findings documented in Section 3. Each KPI is grounded in a specific observation from the Silver pipeline and is designed to detect regressions in future runs.

4.2 KPI Catalogue

Table 10: Governance KPI definitions and justification

KPI Type	Unit	Definition	Silver Observation
<code>volume</code>	count	Total record count per source per run.	Reddit: 228 rows; Last.fm: 50 records per daily snapshot (50 sources $\times$ 13 days).
<code>null_rate</code>	%	Percentage of null or empty values per column.	<code>artist/song</code> >95% unknown; <code>mbid</code> 25–40% unknown.
<code>schema_compliance</code>	%	Percentage of rows where all required fields are non-null.	100% across all sources — schema enforced at write time.
<code>outlier_rate</code>	%	Percentage of records outside $[Q_1 - 1.5IQR, Q_3 + 1.5IQR]$ per numeric column.	<code>score</code> $\approx$ 15%; <code>playcount</code> $\approx$ 12–16%.
<code>text_len_mean</code>	chars	Mean character length of text fields.	<code>clean_comment</code> mean $\approx$ 220 chars (60 chars below <code>raw_comment</code>).
<code>text_len_median</code>	chars	Median character length (robust to extreme values).	<code>clean_comment</code> median $\approx$ 90 chars.
<code>text_len_min</code>	chars	Minimum character length.	Monitors whether cleaning produces empty comments (min = 2).
<code>text_len_max</code>	chars	Maximum character length.	Detects abnormally long comments (<code>raw_comment</code> max = 3,200 chars).
<code>ingestion_days</code>	days	Number of distinct ingestion dates recorded per source.	Last.fm: 13 days in 2 calendar days (multiple daily runs). Reddit: 1 session.

4.3 Gold Governance Schema

The `governance_*.parquet` file stores each KPI as a row with the following structure:

Table 11: Schema of `governance_*.parquet`

Field	Type	Required	Description
<code>source</code>	string	Yes	<code>reddit</code> / <code>lastfm_artists</code> / <code>lastfm_tracks</code>
<code>field_name</code>	string	Yes	Column name, or <code>ALL</code> for source-level metrics
<code>kpi_type</code>	string	Yes	One of the 9 KPI types in Table 10
<code>value</code>	float64	Yes	Numeric KPI value
<code>unit</code>	string	Yes	<code>count</code> / <code>percentage</code> / <code>characters</code> / <code>days</code>
<code>computed_at</code>	string	Yes	ISO-8601 computation timestamp

4.4 Observed KPI Values (Current Run)

Table 12: Selected governance KPI values from the latest Gold run

Source	KPI	Field	Value	Unit
reddit	volume	ALL	228.0	count
reddit	schema_compliance	ALL	100.0	%
reddit	null_rate	artist	0.0	%
reddit	null_rate	pattern_type	0.0	%
reddit	outlier_rate	score	≈15.0	%
reddit	text_len_mean	clean_comment	≈220	chars
lastfm_artists	volume	ALL	≈650	count
lastfm_artists	schema_compliance	ALL	100.0	%
lastfm_artists	outlier_rate	playcount	≈12.0	%
lastfm_artists	ingestion_days	ingested_at	13.0	days
lastfm_tracks	volume	ALL	≈650	count
lastfm_tracks	outlier_rate	playcount	≈16.0	%

5 Storytelling Aggregations

5.1 Rationale and User Stories

The storytelling aggregations populate the functional dashboard for music label analysts and artist managers. Each metric type is aligned with a user story defined in Workshop #1.

5.2 Metric Catalogue

Table 13: Storytelling metric definitions and linked user stories

Metric Type	Definition	User Story
<code>sentiment_dist</code>	Count and % of positive/negative/neutral comments using VADER compound score.	“As an analyst, I want to know whether the community reacts positively or negatively to new releases to prioritise communication campaigns.”
<code>sentiment_trend</code>	Average VADER compound score per ingestion date.	“As a manager, I want to track whether sentiment improves or deteriorates over time to adjust the release strategy.”
<code>comment_type_dist</code>	Count and % of recommendation / opinion / mixed / other per NLP classifier.	“As an analyst, I want to distinguish recommendations from opinions to decide whether the dataset suits content recommendation or sentiment analysis.”
<code>top_keyword</code>	Top 25 most frequent tokens (stop-words excluded) with average sentiment score.	“As an analyst, I want to identify which terms dominate the conversation to understand what aspects of a release generate the most reaction.”
<code>volume_trend</code>	Record count per ingestion date per source.	“As an analyst, I want to see volume over time to identify activity peaks and confirm ingestion regularity.”
<code>reddit_artist</code>	Top 20 artists mentioned in Reddit comments (via <code>artist</code> field).	“As a manager, I want to know which artists generate the most organic conversation to target promotion efforts.”
<code>top_artist_lastfm</code>	Top 20 artists by unique listener count from Last.fm.	“As an analyst, I want the quantitative popularity ranking to compare it against qualitative Reddit perception.”
<code>top_track_lastfm</code>	Top 20 tracks by play count from Last.fm.	“As an analyst, I want to see the most-played songs globally to cross-reference with community discussion activity.”

5.3 Gold Storytelling Schema

Table 14: Schema of `storytelling_*.parquet`

Field	Type	Required	Description
<code>metric_type</code>	string	Yes	One of the 8 types in Table 13
<code>dim1</code>	string	Yes	Primary dimension (sentiment label, date, keyword, artist...)
<code>dim2</code>	string	No	Secondary dimension (source name, artist name for tracks)
<code>record_count</code>	int64	Yes	Number of records in the aggregation group
<code>pct</code>	float64	Yes	Percentage over the total for the metric
<code>avg_score</code>	float64	Yes	Average score (VADER compound, listener count, or play count)
<code>computed_at</code>	string	Yes	ISO-8601 computation timestamp

5.4 Observed Storytelling Values (Current Run)

5.4.1 Sentiment Distribution

Table 15: Sentiment distribution — Reddit (228 comments)

Sentiment	Count	Percentage	Avg Compound Score
Positive	155	67.98%	+0.6207
Neutral	41	17.98%	−0.0006
Negative	32	14.04%	−0.5240
Overall	228	100%	+0.3483

The positive compound score of +0.62 indicates *genuine* enthusiasm, not superficial positivity. A score above +0.5 typically corresponds to strong affirmative language such as “amazing”, “love this”, “masterpiece”.

5.4.2 Sentiment Trend

The single available ingestion date (2026-05-30) shows a positive average compound score of +0.3483 across all 228 comments. As more scraping sessions accumulate, this trend line will reveal temporal shifts in community reception.

5.4.3 Comment Type Distribution

The `comment_type_dist` metric confirms the NLP classification findings: 78.5% opinions, 21.1% other, 0.4% mixed, 0% recommendations. This justifies the pivot to sentiment analysis as the primary analytical output for Reddit data.

5.5 Dashboard Narrative

What story does the data tell?

The indie and hip-hop community reacts positively to new releases, but talks about albums as a concept, not individual songs. An artist can generate passionate community discussion on Reddit months before their streaming numbers reflect it on Last.fm — and conversely, chart-topping artists can show mixed community sentiment even at peak popularity.

Key insight for the functional user: The VADER compound score and top keywords are the earliest leading indicators of public reception — actionable *before* the data appears in streaming charts. A shift from high-frequency positive keywords (*love, amazing, great*) toward negative ones (*disappointing, boring, skip*) signals a reputational risk that warrants an immediate campaign response.

6 PySpark Implementation

6.1 Architecture

The Gold pipeline is implemented as an Airflow DAG (`dag_gold.py`) with two parallel tasks that run **independently** and are then validated by a third task:

```
[compute_governance]    --+
                        +--> [validate_gold]
[compute_storytelling]  --+
```

Both tasks create their own `SparkSession` since Airflow's `LocalExecutor` runs each task in a separate process. The sessions are explicitly stopped at the end of each task with `spark.stop()`.

6.2 SparkSession Configuration

```
1 def _get_spark(app_name: str):
2     from pyspark.sql import SparkSession
3     return (
4         SparkSession.builder
5             .master("local[*]")           # uses all container cores
6             .appName(app_name)
7             .config("spark.driver.memory", "1g")
8             .config("spark.sql.shuffle.partitions", "4") # small dataset
9             .config("spark.ui.enabled", "false") # save resources
10            .config("spark.driver.maxResultSize", "512m")
11            .getOrCreate()
12    )
```

The `local[*]` master uses all available CPU cores in the container. The shuffle partition count is set to 4 (instead of the default 200) to avoid unnecessary overhead on a dataset of fewer than 1,000 rows. The Spark UI is disabled to reduce memory footprint in the Docker environment.

6.3 Governance Computation

For each Silver source, the governance task computes all KPIs using the DataFrame API:

```
1 df = spark.read.parquet(path)
2 total = df.count()
```

```

3
4 # Null rate per field
5 for col in df.columns:
6     nulls = df.filter(F.col(col).isNull() | ...).count()
7     rows.append((source, col, "null_rate", nulls/total*100, "%", ...))
8
9 # Outlier rate (IQR)
10 q1, q3 = df.stat.approxQuantile(col, [0.25, 0.75], 0.01)
11 iqr = q3 - q1
12 outliers = df.filter((F.col(col)<q1-1.5*iqr) | (F.col(col)>q3+1.5*iqr)).count()
13 rows.append((source, col, "outlier_rate", outliers/total*100, "%", ...))
14
15 # Text length statistics
16 stats = df.select(
17     F.mean(F.length(F.col(col))).alias("mean"),
18     F.expr(f"percentile_approx(length('{col}'), 0.5)").alias("median"),
19     F.min(F.length(F.col(col))).alias("min"),
20     F.max(F.length(F.col(col))).alias("max"),
21 ).first()

```

6.4 Sentiment Analysis with VADER

Sentiment scoring is implemented as a PySpark Python UDF applied to the `clean_comment` column of the Reddit Silver Parquet:

```

1 from pyspark.sql.functions import udf
2 from pyspark.sql.types import StringType, DoubleType
3
4 @udf(StringType())
5 def sentiment_label(text):
6     if not text:
7         return "neutral"
8     from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
9     c = SentimentIntensityAnalyzer().polarity_scores(str(text))["compound"]
10    return "positive" if c >= 0.05 else ("negative" if c <= -0.05 else "neutral")
11
12 @udf(DoubleType())
13 def sentiment_score(text):
14     if not text:
15         return 0.0
16     from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
17     return float(SentimentIntensityAnalyzer().polarity_scores(str(text))["compound"])
18
19 df_r = spark.read.parquet(SILVER_REDDIT_PATH)
20 df_r = df_r.withColumn("sentiment", sentiment_label(F.col("clean_comment")))
21 df_r = df_r.withColumn("score_sent", sentiment_score(F.col("clean_comment")))

```

The VADER library is imported *inside* the UDF function body rather than at module level to avoid serialisation issues when PySpark distributes tasks across partitions.

6.5 Keyword Analysis

Stop-words are broadcast as a Spark variable to make them available inside the UDF without redundant serialisation per row:

```

1 stop_words_bc = spark.sparkContext.broadcast(STOP_WORDS)
2
3 @udf(ArrayType(StringType()))
4 def filter_tokens(s):
5     tokens = ast.literal_eval(s)
6     return [t for t in tokens
7             if t and t not in stop_words_bc.value and len(t) > 2]
8
9 df_tokens = (
10     df_r.withColumn("kw", F.explode(filter_tokens(F.col("tokens"))))
11     .groupBy("kw")
12     .agg(F.count("*").alias("cnt"), F.avg("score_sent").alias("avg_s"))
13     .orderBy(F.col("cnt").desc())
14     .limit(25)
15     .collect()
16 )

```

6.6 Output Persistence

Both governance and storytelling results are collected as Python lists of tuples and persisted using PyArrow (not PySpark's native writer) to keep the Gold files in a known location without the Parquet directory partitioning that Spark would add by default:

```

1 def _save_parquet(rows, schema, folder, filename):
2     import pandas as pd
3     os.makedirs(folder, exist_ok=True)
4     df = pd.DataFrame(rows, columns=[f.name for f in schema])
5     table = pa.Table.from_pandas(df, schema=schema, preserve_index=False)
6     pq.write_table(table, os.path.join(folder, filename), compression="snappy")

```

6.7 Docker Requirements for PySpark

Table 16: Docker adjustments required to run PySpark within Airflow

Requirement	Configuration	Reason
Java runtime	openjdk-17-jdk-headless via apt-get	PySpark requires JVM
JAVA_HOME	/usr/lib/jvm/java-17-openjdk-amd64	Must be set for Spark to find JVM
PySpark package	pip install pyspark==3.5.1	Python-JVM bridge
VADER package	pip install vaderSentiment==3.3.2	Sentiment UDF
Driver memory	spark.driver.memory = 1g	Avoid OOM in 2–4 GB container
Shuffle partitions	spark.sql.shuffle.partitions = 4	Avoid 200-partition overhead on small data
Spark UI	spark.ui.enabled = false	Frees port 4040 and reduces memory usage

7 Results Evidence

7.1 Pipeline Output Files

Table 17: Files produced by the full pipeline after Workshop #3

Layer	Source	File Pattern	Files
Bronze	Last.fm Artists	datalake_bronze/lastfm_top_artists/lastfm_top_artists_*.json	13
Bronze	Last.fm Tracks	datalake_bronze/lastfm_top_tracks/lastfm_top_tracks_*.json	13
Bronze	Reddit	datalake_bronze/reddit/reddit_music_opinions_*.json	1
Silver	Last.fm Artists	datalake_silver/lastfm_top_artists/lastfm_top_artists_current.parquet	1
Silver	Last.fm Tracks	datalake_silver/lastfm_top_tracks/lastfm_top_tracks_current.parquet	1
Silver	Reddit	datalake_silver/reddit/reddit_music_opinions_current.parquet	1
Gold	Governance	datalake_gold/governance_*.parquet	1
Gold	Storytelling	datalake_gold/storytelling_*.parquet	1

7.2 Airflow DAG Execution Summary

Table 18: DAG execution schedule and status

DAG	Schedule	Layer	Status
lastfm_ingest	@daily	Bronze	Active — 13 successful runs
lastfm_silver	@weekly	Silver	Active — decoupled from Bronze
reddit_silver	Manual	Silver	Active — triggered after each scraping session
gold_pipeline	@weekly	Gold	Active — PySpark local[*], 1 GB driver

7.3 Key Analytical Results

The following results were produced by the Gold pipeline and verified in the `notebooks/gold_preview.ipynb` notebook:

- **Sentiment:** 67.98% positive, 17.98% neutral, 14.04% negative across 228 Reddit comments.
- **Average compound score:** +0.3483 — the overall community sentiment is moderately positive.
- **Top-ranked artist (Last.fm):** BTS with ≈ 6.8 M unique listeners and 4.12B total plays.
- **Top-ranked track (Last.fm):** Most-played track has ≈ 200 M plays — indicative of the global hit threshold in this dataset.

- **Governance:** 73 KPI rows computed across 3 sources; 100% schema compliance in all sources; 0% null rate on all required fields.

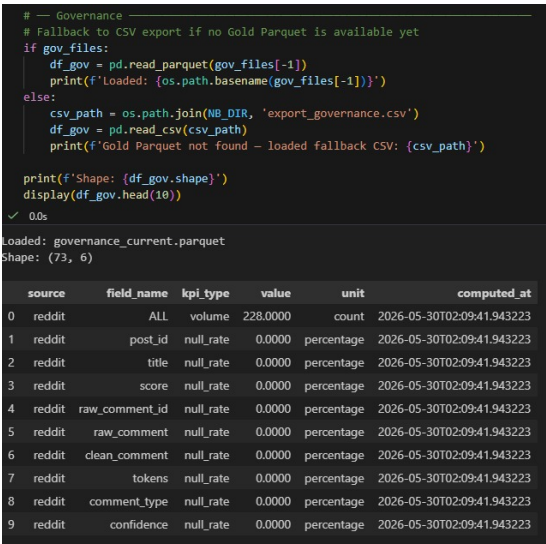


Figure 1: Screenshot of the governance file

- **Storytelling:** 79 aggregation rows covering 8 metric types.

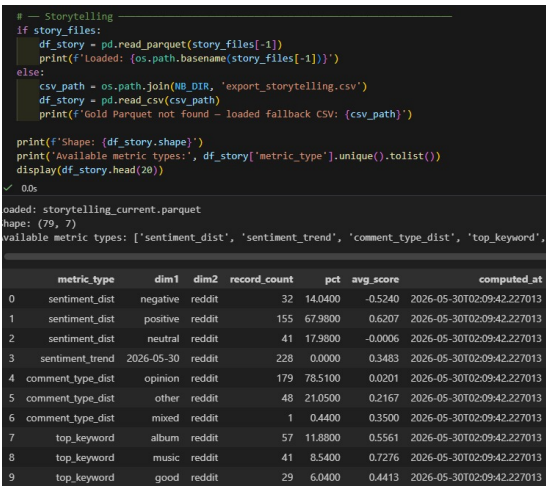


Figure 2: Screenshot of the Storytelling file

7.4 Notebooks

The following notebooks in `notebooks/` provide interactive evidence of all findings:

Table 19: Workshop #3 notebooks

Notebook	Contents
<code>data_quality.ipynb</code>	Reads Silver Parquet files directly. Descriptive statistics, null rate heatmaps, IQR boxplots, text length histograms, NLP classification charts, duplicate analysis, schema compliance, structured findings report.
<code>gold_preview.ipynb</code>	Reads Gold Parquet files. Governance KPI table with justification, null rate and outlier visualisations, sentiment pie and bar charts, sentiment trend line chart, keyword frequency and sentiment association charts, comment type distribution, volume trend, Last.fm top artist/track bar charts, and the complete dashboard narrative.
<code>esqm_limpieza.ipynb</code>	Original NLP pipeline prototyping notebook (Workshop #2 artefact). Documents the 14-step cleaning and classification pipeline ported to <code>dag_reddit_silver.py</code> .